

Guia de Apresentação e Explicação das Questões

— Computação Gráfica

Este documento é um roteiro completo, amigável e direto ao ponto para explicar cada questão da avaliação perante o professor e os colegas, sem deixar de lado nenhum detalhe técnico ou matemático importante.

1. Introdução Geral (30 segundos)

“Professor, organizamos nosso projeto em C++ com OpenGL clássico e FreeGLUT, utilizando CMake como sistema de build. Dividimos o trabalho em três executáveis independentes — um para cada questão da prova — além de uma suíte de testes automatizados (`./build/test_matematica`) que valida todas as fórmulas, octantes e matrizes antes mesmo de abrir a interface gráfica.”

2. Questão A — Traçado de Retas (3,0 pontos)

Executável: `./build/questao1_retas`

2.1 O “Pulo do Gato” (Aviso Inicial)

*“O ponto principal da nossa implementação é que **não usamos** `glBegin(GL_LINES)`. Se usássemos `GL_LINES`, estaríamos delegando a discretização dos pixels para a GPU, o que anularia o objetivo da questão. Nós calculamos cada coordenada inteira (x, y) diretamente na CPU e desenhamos pixel por pixel via `GL_POINTS`, imprimindo a lista completa no terminal.”*

2.2 Como Explicar os 3 Algoritmos

1. Equação Analítica da Reta ($y = mx + b$)

- **Conceito:** A abordagem direta da geometria analítica. Calculamos a inclinação $m = \frac{\Delta y}{\Delta x}$ e o coeficiente linear $b = y_1 - m \cdot x_1$.
- **Cuidado no código:** Se a reta for íngreme ($|m| > 1$), iterar no eixo X pula vários valores de Y , criando “buracos”. Portanto, invertemos o eixo condutor: iteramos em Y e calculamos $x = \text{round}\left(\frac{y-b}{m}\right)$. Se for vertical ($\Delta x = 0$), travamos X e variamos Y .
- **Limitação:** Realiza multiplicações e divisões com números reais (`float`) e depende da função `round()` a cada iteração, sendo computacionalmente pesado.

2. DDA (*Digital Differential Analyzer*)

- **Conceito:** Explora a propriedade incremental da reta (equação de diferenças finitas). Em vez de recalculer a reta do zero a cada ponto, ele apenas soma um passo constante.
- **Mecanismo:** Descobre quem tem maior variação ($\text{steps} = \max(|\Delta x|, |\Delta y|)$) e calcula os incrementos fracionários:

$$\Delta x_{\text{inc}} = \frac{\Delta x}{\text{steps}}, \quad \Delta y_{\text{inc}} = \frac{\Delta y}{\text{steps}}$$

- **Na iteração:** Apenas acumula: $x_{k+1} = x_k + \Delta x_{\text{inc}}$ e plota com `round()`.
- **Vantagem vs Limitação:** Substitui multiplicações por somas sucessivas (mais rápido), mas ainda usa ponto flutuante e chamadas a `round()`.

3. Bresenham (Aritmética Inteira Pura)

- **Conceito:** O padrão ouro da computação gráfica. Jack Bresenham percebeu que telas são malhas discretas de inteiros. Logo, não faz sentido usar números de ponto flutuante no laço principal.
- **Mecanismo:** Mantém uma variável de decisão inteira inicial ($\text{err} = \Delta x - \Delta y$). A cada iteração:
 - Avalia o dobro do erro ($e2 = 2 \cdot \text{err}$).
 - Se $e2 > -\Delta y$, avança um passo em X e ajusta o erro.
 - Se $e2 < \Delta x$, avança um passo em Y e ajusta o erro.
 - Se ambas as condições forem atendidas, avança em diagonal (8-conectividade perfeita).
- **Por que é superior:** Multiplicações por 2 são feitas por deslocamento de bits ($\ll 1$), e o laço só executa adições e subtrações com inteiros, tornando-o extremamente rápido e implementável em silício/hardware simples.

2.3 Perguntas Prováveis da Banca

- **Pergunta:** “*Por que Bresenham é melhor que o DDA para hardware?*”
Resposta: O DDA exige acumuladores de ponto flutuante e instruções de arredondamento (`round()`), exigindo uma FPU (Floating-Point Unit). O Bresenham utiliza apenas lógica inteira e somas/subtrações, rodando com velocidade máxima até em microcontroladores simples.
- **Pergunta:** “*O algoritmo de vocês funciona em qualquer direção?*”
Resposta: Sim. O algoritmo trata os passos sx e sy (± 1) de acordo com o sinal das diferenças, cobrindo todos os 8 octantes sem furos.

3. Questão B — Transformações Geométricas 3D (3,0 pontos)

Executável: `./build/questao2_transformacoes`

3.1 O “Pulo do Gato” (A Inconsistência do Enunciado)

“Ao analisar os quatro pontos da base dados na prova (A, B, C, D) , calculamos o **produto misto** dos vetores: $(A-D) \cdot ((B-D) \times (C-D))$. O resultado obtido é **1000**, e não zero. Isso prova que os 4 pontos não formam um plano perfeito no espaço 3D original.” “Como nossa diretriz foi manter total fidelidade aos dados da prova sem alterar coordenadas, mantivemos os vértices originais e utilizamos o **Vetor Normal de Newell** para encontrar o plano médio ótimo da base e alinhá-lo perfeitamente ao plano XZ .”

3.2 A Sequência de Transformações

1. **Requisito 1: Vértice D na origem $(0, 0, 0)$**
 - Coordenadas de $D = (5, 0, -25)$.
 - Aplicamos a translação oposta: $\vec{T} = (-5, 0, 25)$.
 - No OpenGL: `glTranslatef(-5.0f, 0.0f, 25.0f)`.
2. **Requisito 2: Base $ABCD$ paralela ao plano XZ**
 - O vetor normal ao plano XZ é o eixo Y (direção vertical).
 - Calculamos o vetor normal médio da base usando a fórmula de Newell (área das projeções):

$$\vec{N} = (-200, 1000, 0) \implies \hat{n} \approx (-0.1961, 0.9806, 0.0)$$

- Determinamos o eixo de rotação pelo produto vetorial com o eixo Y e o ângulo pelo produto escalar ($\theta \approx 168.69^\circ$).
3. **Requisito 3: Vértice E abaixo da base com Y negativo**
 - Ao rotacionar a base para ficar horizontal, poderíamos escolher a normal apontando para cima ($+Y$) ou para baixo ($-Y$).
 - Se alinhássemos a normal com $+Y$, o ápice E apontaria para cima ($Y_E > 0$).
 - Por isso, alinhamos a normal deliberadamente com $-Y$ (**para baixo**).
 - Resultado: a base fica apoiada no plano XZ e o ápice termina em $Y_E = -17.65 < 0$, cumprindo rigorosamente a condição da prova.

3.3 Demonstração Visual

- Pressione **O** para exibir o sólido **Original** (cinza).
- Pressione **T** para exibir o sólido **Transformado** (verde).
- Pressione **A** para exibir **Ambos** simultaneamente.
- Arraste o mouse com o botão esquerdo para girar a câmera 3D e mostrar o vértice D na origem e o topo virado para baixo.

4. Questão C — Projeções Geométricas (4,0 pontos)

Executável: `./build/questao3_projecoos`

4.1 O “Pulo do Gato” (Implementação Manual das Matrizes)

*“Professor, todas as projeções foram calculadas manualmente via álgebra linear homogênea 4×4 no nosso código C++. Não utilizamos funções prontas de projeção da GPU como `gluPerspective` ou `glFrustum`. Nosso código multiplica cada vértice pela matriz homogênea, calcula o fator w e realiza a **divisão homogênea** antes de desenhar em 2D.”*

4.2 Explicação das Projeções Canônicas

1. Projeção Paralela Ortográfica (Tecla 1)

- **Conceito:** Raios de projeção paralelos entre si e perpendiculares à tela (XY).
- **Matemática:** A matriz simplesmente descarta o eixo Z ($z' = 0$). As distâncias e o paralelismo são estritamente preservados; não há distorção de perspectiva.

2. Projeções Oblíquas: Cavaleira vs Cabinet (Teclas 2 e 3)

- **Conceito:** Os raios de projeção continuam paralelos, mas incidem de forma oblíqua no plano de projeção. O plano frontal (XY) mantém suas medidas reais, enquanto o eixo de profundidade (Z) é inclinado em um ângulo α (convencionalmente 45°).
- **Cavaleira** ($l = 1.0$): O fator de escala de profundidade é mantido em **100%**. Gera um efeito alongado/esticado.
- **Cabinet** ($l = 0.5$): O fator de escala de profundidade é **reduzido pela metade (50%)**. Produz um efeito visualmente mais proporcional e confortável ao olho humano.

3. Projeções em Perspectiva: 1, 2 e 3 Pontos de Fuga (Teclas 4, 5 e 6)

- **Como a perspectiva surge na matriz:**
 - O segredo está nos termos da **última linha da matriz** 4×4 :

$$\begin{bmatrix} -\frac{1}{f_x} & -\frac{1}{f_y} & -\frac{1}{f_z} & 1 \end{bmatrix}$$

- Após a multiplicação matricial, o fator homogêneo w passa a depender da posição do ponto:

$$w = 1 - \frac{x}{f_x} - \frac{y}{f_y} - \frac{z}{f_z}$$

- Em seguida, dividimos as coordenadas por w ($x_p = \frac{x}{w}, y_p = \frac{y}{w}$). Pontos mais distantes geram um w maior e, portanto, **ficam menores na tela**, reproduzindo a visão humana!

- **1 Ponto de Fuga (Tecla 4):** Foco apenas no eixo Z . As arestas de profundidade convergem para um único ponto no horizonte (efeito trilho de trem/túnel).
- **2 Pontos de Fuga (Tecla 5):** Focos nos eixos X e Z . Efeito clássico de desenho arquitetônico: observar uma esquina ou quina de edifício.
- **3 Pontos de Fuga (Tecla 6):** Focos nos eixos X , Y e Z . Perspectiva aérea (“olho de pássaro”) ou de baixo para cima (“olho de minhoca”), onde até as retas verticais convergem.

4.3 Demonstração no Terminal e Janela

- Pressione a tecla **C** para trocar o sólido em tempo real entre o **Cubo**, a **Casinha 3D** e o próprio **Poliedro da Questão 2**.
- Mostre no terminal que a cada tecla (1 a 6), o programa imprime a matriz 4×4 exata utilizada e a tabela com as coordenadas transformadas antes e depois da divisão por w .

5. Tabela Resumo “De Bolso”

Questão	Conceito-Chave	Argumento Rápido de Apresentação
Q1 — Retas	GL_POINTS e Bresenham	<i>“Rasterização feita na CPU ponto a ponto; Bresenham é puro cálculo inteiro com shifts de bits, sem usar floats nem round().”</i>
Q2 — Transformações	Produto Misto & Newell	<i>“A base da prova não é perfeitamente coplanar (produto misto = 1000); usamos o vetor médio de Newell e alinhamos a normal a $-Y$ para garantir $Y_E < 0$.”</i>
Q3 — Projeções	Matriz 4×4 e Divisão por w	<i>“Matrizes homogêneas puras calculadas no C++. A perspectiva surge naturalmente na divisão homogênea das coordenadas por w.”</i>